

W3大地图深度烘焙与伪3D渲染工作流 副本

来源：W-Game 团队 | 案例规模：解决方案 | 上线时间：2026.05.06

一、导言

问题背景

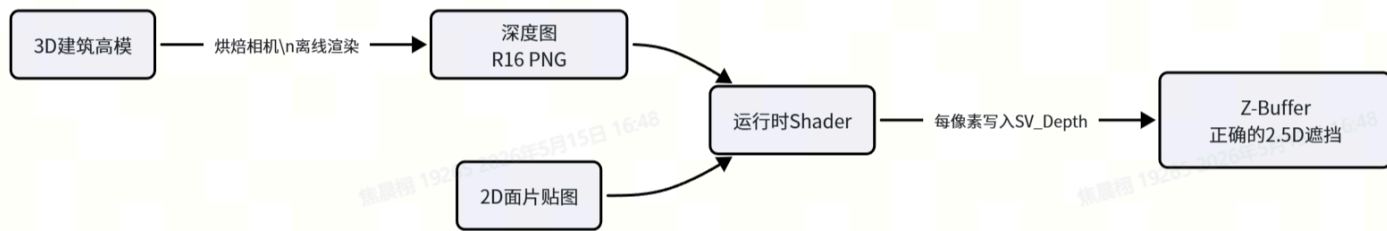
SLG 游戏大地图中存在海量建筑与物件：

- **全部使用 3D 模型** → 顶点数量和 DrawCall 严重超标
- **全部降维成纯 2D 贴图（Quad 面片）** → 丧失与场景中角色、特效的 Z 轴（深度）遮挡关系，导致视觉穿帮



技术解法：深度伪造（Depth Imposter）

💡 **核心思路：**在渲染平面 2D 贴图时，读取预先烘焙好的深度图（Depth Texture），在片元着色器（Fragment Shader）中强行修改像素的深度值（`SV_Depth`），从而让单薄的面片在 Z-Buffer 中"拥有"真实的 3D 体积。



管线收益：用极低的片元计算开销，换取海量的顶点节省，实现完美的 2.5D 深度融合。

二、整体设计框架

大地图建筑的核心矛盾拆成两个阶段：

离线阶段（烘焙）

用一个专用的渲染相机，把 3D 建筑的深度信息压进一张与 2D 面片 UV 对齐的 16 位灰度图（R16 PNG）。每个像素记录的是：该像素位置对应的 3D 表面，在深度方向上距离面片前表面有多远。

运行时（Shader）

渲染 2D 面片时，对每个像素从深度图采样，把烘焙时记录的深度信息换算成当前帧的 NDC 深度值，写入 `SV_Depth` 输出。GPU 拿到这个深度后，Z-Buffer 测试的行为就跟 3D 网格完全一样了——建筑后面的角色被遮住，建筑前面的特效正确叠在上面。

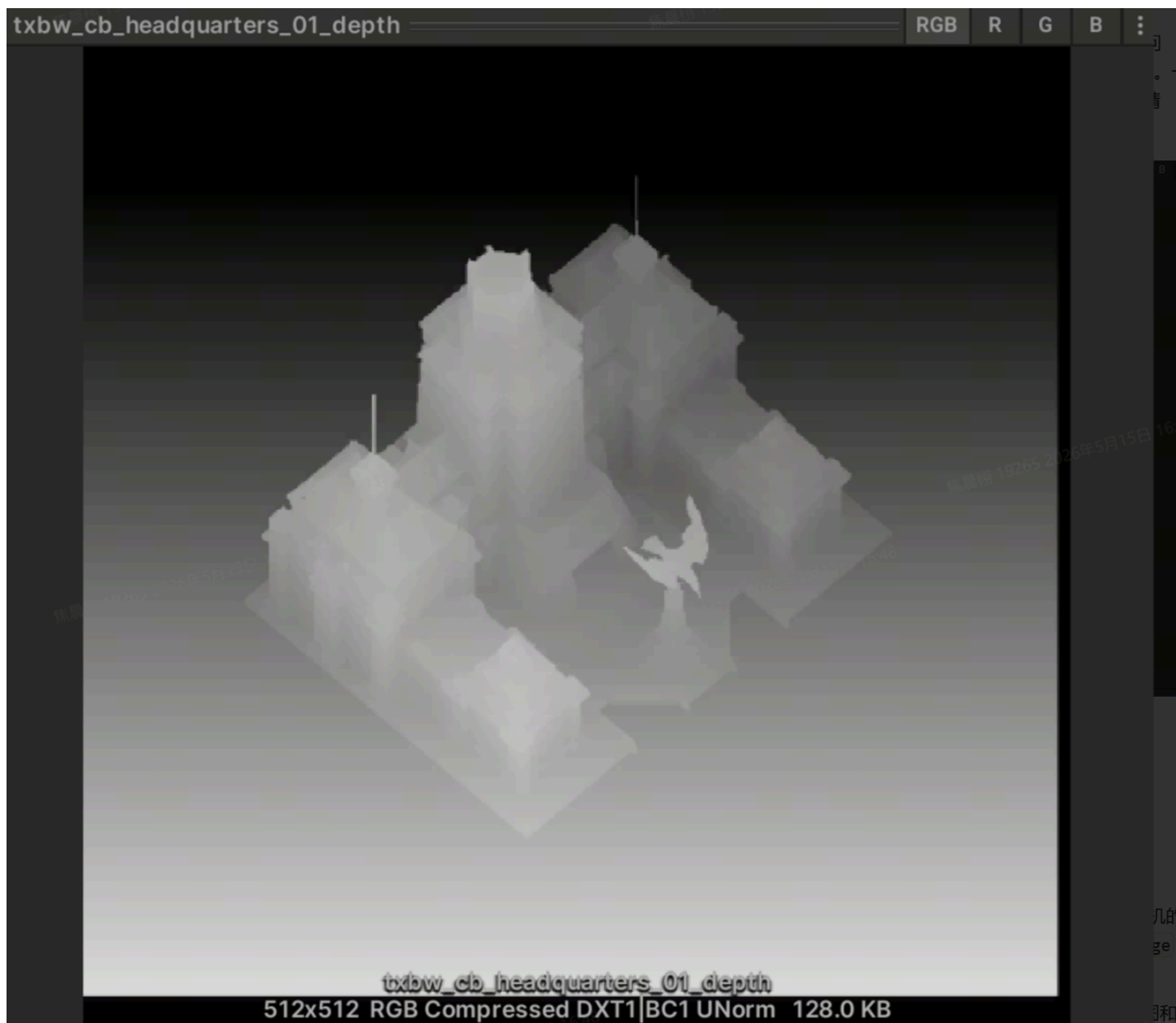
两个核心工程问题

- 1. **深度图里存什么：**不同的存储方式决定了运行时的换算复杂度和对缩放变化的容忍度
 - 2. **UV 如何对齐：**深度图和 2D 面片贴图必须像素对齐，否则深度关系会出现倾斜和错位
- 两套方案（Camera Space / Object Space）分别对应内城和外城的使用场景。

三、方案一：相机空间映射（Camera Space）

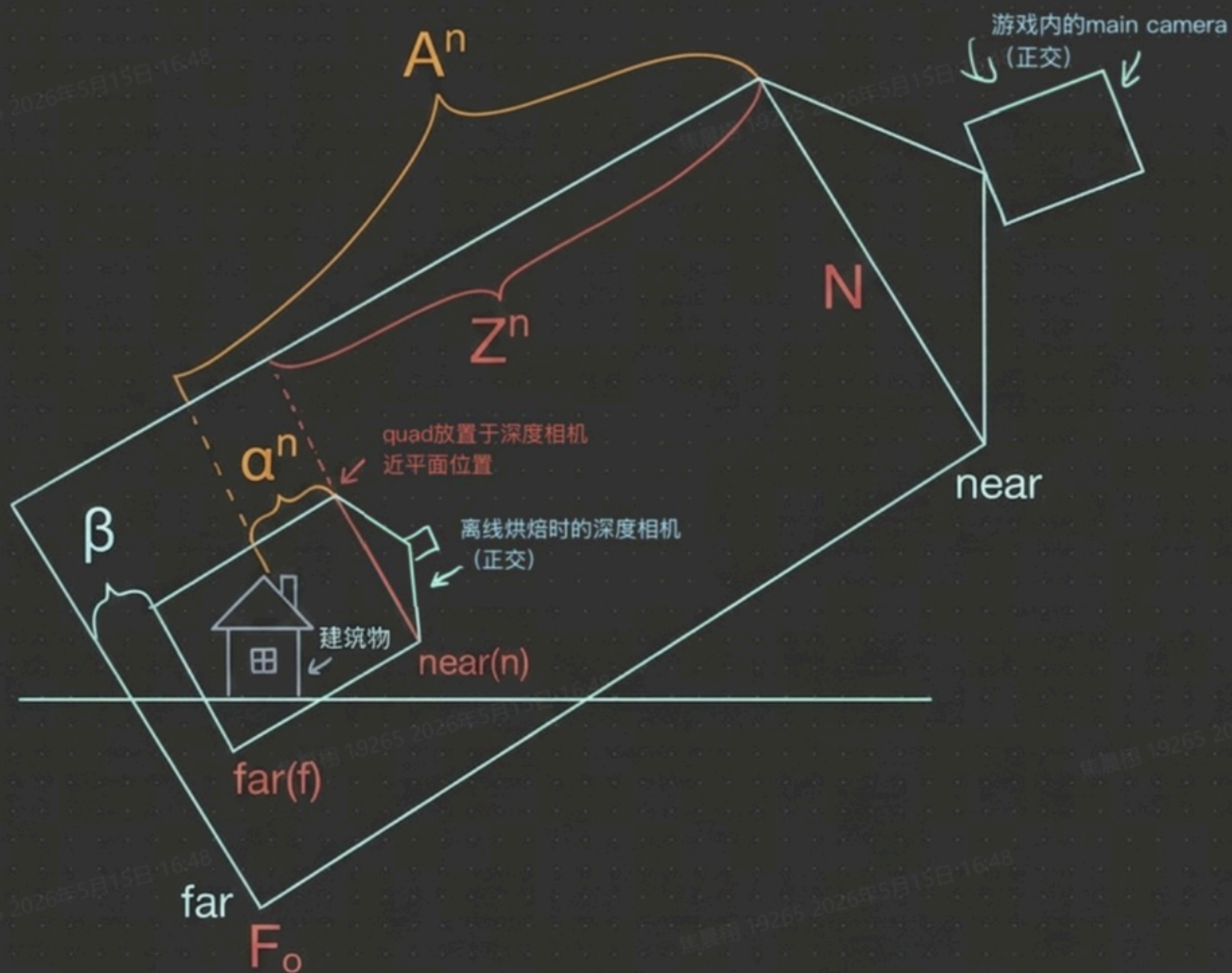
定位：大地图渲染的性能基石。极低 ALU 开销，适用于正交视角下海量、静态、无非均匀缩放的背景建筑。

为什么需要专用烘焙相机



直接用主相机截一帧深度 Buffer 存成贴图的致命问题：**精度极差**。

- 主相机近远平面覆盖整张大地图深度范围：Near ≈ 5 ，Far ≈ 1100 ，差值约 1100 单位
- 一栋建筑的 Z 向厚度可能只有 30-60 单位
- 16 位精度分配给 1100 范围，建筑只占约 5% $\rightarrow$ 大量阶梯噪声，锯齿分层



$$\beta = \frac{(F-N) - (f-n) - (1-z^n) \times (F-N)}{F-N} \quad (\text{vertex shader})$$

$$A^n = \beta + \alpha^n \times \frac{f-n}{F-N} \quad (\text{fragment shader})$$

✅ **解决方案：**设计专用烘焙相机：

- 与游戏主相机方向完全一致（正交，同一视角）
- 近远平面紧贴建筑：Near $\approx$ 5，Far $\approx$ 65，差值约 60 单位
- 16 位精度完整分配给 60 单位，精度提升约 18 倍

⚠️ **代价：**运行时 Shader 需要知道烘焙相机参数（`DepthCameraRange` 和 `MainCameraRange`），才能将深度图的值正确还原回主相机 NDC 空间。

附带收益：烘焙相机精确控制烘焙视口分辨率（自动匹配 Sprite 贴图尺寸），保证深度图和 2D 面片贴图天然像素对齐。

核心公式

代码块

```
1   $A^n = \beta + a^n \times (f-n) / (F-N)$ 
2
```

数学前提：正交相机下，NDC Z 值与视图空间距离是线性关系。正交投影的变换矩阵只有平移和缩放，没有透视除法，所以 view-Z 到 NDC-Z 是线性的，可以直接做比例换算。透视相机不满足此条件（view-Z 与 NDC-Z 是倒数关系）。

Shader 实现

Vertex 阶段 — 计算基准深度 β ：

代码块

```
1  // Reversed-Z (DX11): clipSpaceDepth 范围 [1→0]，1=近，0=远
2  float clipSpaceDepth = output.positionCS.z;
3  float normalizedDepth = 1 - clipSpaceDepth; // 转为标准方向：0=近，1=远
4
5  output.mainDepth = (mainCameraRange - depthCameraRange
6                      - normalizedDepth * mainCameraRange) / mainCameraRange;
7
```

Fragment 阶段 — 叠加深度图偏移（仅 1 条 multiply-add）：

代码块

```
1  half depthValue = SAMPLE_TEXTURE2D(_DepthTex, sampler_DepthTex, uv).r;
2  outputDepth = depthValue * (depthCameraRange / mainCameraRange) + mainDepth;
3
```

图形 API 适配：

代码块

```
1  #ifdef UNITY_REVERSED_Z
2      // Direct3D, Vulkan, Metal: reverse Z (1 = near, 0 = far)
3      depth = i.mainDepth - (1 - tex2D(_DepthTex, i.uv).x) * (_DepthCamRng /
4                      _MainCamRng);
5      depth += _DepthOffset;
```



```
5  #else
6      // OpenGL: standard Z (0 = near, 1 = far)
7      depth = i.mainDepth + (1 - tex2D(_DepthTex, i.uv).x) * (_DepthCamRng /
    _MainCamRng);
8      depth -= _DepthOffset;
9  #endif
10
```

四、方案二：物体空间映射（Object Space）

定位：基于本地坐标，根据离线深度信息在运行时重建物体的法线和物理尺度，再乘以变换矩阵还原世界空间下的深度。

为什么需要 Object Space

Camera Space 的根本限制：**烘焙时把缩放信息固化进了深度图**。美术调整缩放后深度图与面片空间映射错误，必须重烘焙。外城建筑美术需要频繁调整面片缩放，Camera Space 重烘焙成本不可接受。

✅ **Object Space 思路**：深度图存相对深度（以 quad 自身尺寸为参照的无量纲比例），运行时 Shader 实时计算当前面片的世界空间法线和物理尺度，再还原出真实世界偏移量。美术调整面片缩放时，Shader 里的 `scaleWS` 自动跟随变化，**无需重烘焙**。

⚠️ **代价**：运行时 ~30 条 ALU（vs Camera Space 的 1 条），仅用于外城核心建筑，不铺量。

深度图编码

深度图存储"该像素对应 3D 表面点在 quad 本地坐标系 Z 轴上的归一化位置"，单位是 quad 的 XY 物理平均尺寸（RefScale）。

运行时解码：

代码块

```
1  float normalizedZ = SAMPLE_TEXTURE2D(_DepthTex, sampler_DepthTex, uv).r;
2  float localZN      = _LocalZMin + normalizedZ * (_LocalZMax - _LocalZMin);
3  // localZN 的单位是 RefScale，与绝对场景缩放无关
4
```

🧠 核心算法：ddx/ddy 实时重建法线与物理尺度

Quad 面片没有切线/副切线顶点属性，传统切线空间方法不适用。借助 GPU 屏幕空间偏导数 **ddx/ddy** 来求逆 Jacobian 矩阵：

代码块

```

1 // 获取相邻像素的世界坐标差和 UV 差
2 float3 dpdx = ddx(input.posWS);
3 float3 dpdy = ddy(input.posWS);
4 float2 duvdx = ddx(input.uv);
5 float2 duvdy = ddy(input.uv);
6
7 // 求逆 Jacobian, 得到切线/副切线 (沿 U/V 方向的世界空间变化向量)
8 float det = duvdx.x * duvdy.y - duvdx.y * duvdy.x;
9 float invDet = (det >= 0.0 ? 1.0 : -1.0) / max(abs(det), 1e-12);
10 float3 dPdu = ( dpdx * duvdy.y - dpdy * duvdx.y) * invDet;
11 float3 dPdv = (-dpdx * duvdy.x + dpdy * duvdx.x) * invDet;
12
13 // 叉乘得到法线, 确保朝向相机侧
14 float3 axisWS = normalize(cross(dPdu, dPdv));
15 if (dot(axisWS, normalize(_WorldSpaceCameraPos - input.posWS)) < 0.0)
16     axisWS = -axisWS;
17
18 // 切线副切线长度 = 1 单位 UV 对应的世界物理长度
19 // 几何平均得到 scaleWS, 即 RefScale 当前在世界空间中的实际大小
20 float scaleWS = sqrt(max(length(dPdu) * length(dPdv), 1e-12));
21

```

从本地 Z 到 SV_Depth

代码块

```

1 // localZN (单位 RefScale) * scaleWS (RefScale 当前世界大小) = 真实世界偏移量
2 // 🎯 关键: 无论面片被缩成什么大小, 都能还原出正确的世界空间偏移
3 float3 posWS_new = input.posWS + axisWS * (localZN * scaleWS);
4
5 // 重投影到裁剪空间, 透视除法得到 NDC Z
6 float4 posCS_new = TransformWorldToHClip(posWS_new);
7 #ifdef UNITY_REVERSED_Z
8     depth = posCS_new.z / posCS_new.w + _DepthOffset;
9 #else
10     depth = posCS_new.z / posCS_new.w * 0.5 + 0.5 - _DepthOffset;
11 #endif
12

```

五、性能评估

Camera Space

-  优势：Fragment 阶段指令极少，规避了所有向量与矩阵运算，移动端极度友好
-  约束：一旦主相机切透视或物件缩放，线性映射立即失效导致穿帮

Object Space

-  计算包含大量 `cross`、`normalize`、`sqrt`，相对 Camera Space 有明显增量（~30 vs ~1 ALU）
-  `ddx/ddy` 强制唤醒 2×2 像素块（Helper Lanes），需避免重叠时 Overdraw
-  在 W3 中仅用于大地图核心建筑（基地皮肤）

详细对比

指标	Camera Space	Object Space
Fragment ALU	~1	~30
Helper Lane 需求	无	2×2
Blend	One Zero	One Zero
ZWrite	On	On
ZTest	LEqual	LEqual
SV_Depth 禁用 Early-Z	两者共同	两者共同
实机帧率（vivo Y31s, 60 个全屏）	59-60 fps	59-60 fps
W3 应用场景	内城建筑物（海量铺设）	大地图皮肤（核心建筑）
缩放支持		

六、配套渲染工具与 workflow

设计目标

让美术的烘焙操作简化为：**摆好模型和面片 → 点击 Bake**。所有需要精确计算和坐标系对齐的环节全部自动处理。

渲染层过滤与数据清洗

第一轮：保留可绘制的静态网格，剔除粒子系统、特效、`SkinnedMeshRenderer` 等不参与静态深度烘焙的组件，避免包围盒污染 Z 极值。

第二轮：排除目标 2D 面片自身，防止面片的包围盒干扰 3D 高模的实际 Z 边界。

坐标系对齐

计算 ZMin/ZMax 时 **不直接在世界空间取 Min/Max Z 值**，而是先用 quad 的

`worldToLocalMatrix` 把所有 Renderer AABB 的 8 个角点变换到 quad 本地坐标系，再取 Z 极值。

意义：烘焙时的深度采样方向和运行时 Shader 用 `axisWS` 推入的方向必须来自同一个坐标系。用世界空间 Z 值会导致建筑旋转时坐标系不对齐，深度梯度倾斜。

ZMin/ZMax 自动计算与材质回写

- 烘焙时自动计算并直接写入目标面片的 `sharedMaterial`（作用于材质球资产）
- 对应材质属性标注 `[HideInInspector]`，Inspector 不可见，避免美术误操作
- **整个流程对美术透明**——烘焙完成即已配置好

分辨率与 UV 自动对齐

1. **Game View 分辨率自动匹配：**通过反射 API 访问 `UnityEditor.GameView` 内部接口，将 Game View 分辨率切换为当前 Sprite 贴图像素尺寸
2. **相机视口自动适配：**将 Sprite 的 4 个本地角点变换到世界空间，投影到相机平面后取边界，精确覆盖 Sprite 世界范围

七、优化展望

1. **贴图通道合并：**Albedo 放 RGB 通道，Depth 烘焙到 Alpha 通道，一次 Texture Sample 同时拿到颜色和深度，减少一次采样带宽
2. **SRP Batcher 进一步优化：**通过工具强制规范材质变体（Variants），进一步压低大地图的 SetPass Call

八、架构总结

通过 "**Camera Space 的极致算力压榨**" 与 "**Object Space 的高度灵活包容**"，配合底层严格的数据防呆工具链，构建了一套健壮的 W3 正交大地图深度渲染管线。

两套方案的决策逻辑：

- 🏠 **内城建筑**（固定缩放、海量铺设）→ Camera Space
- 🏢 **外城核心建筑**（频繁调整缩放）→ Object Space

工具链统一，美术操作一致，运行时无缝切换。